Table of Contents

List of Figures

List of Tables

1 INTRODUCTION

1.1 Background

People spend a large proportion of their time indoors. Although indoor spaces are often considered safe, they can contain pollution sources, including cooking, cleaning products, aerosol sprays, and inadequate ventilation. The United States Environmental Protection Agency states that indoor air-quality problems are often caused by indoor sources that release gases or particles, while poor ventilation can allow pollutants to accumulate indoors [1].

Particulate matter is one of the main pollutants considered in this project. Fine particles can remain suspended in air for long periods and can be inhaled deep into the respiratory system. The World Health Organization identifies exposure to fine particulate matter as a major environmental health risk [2]. Cooking is a common indoor source of particulate matter, particularly during frying, heating, or food preparation. Even when an induction hob is used, cooking can still produce fine particles that may affect indoor air quality over time [3].

Volatile organic compounds (VOCs) are also relevant because they can be released from household products such as air fresheners, paints, sprays, and cleaning products. The EPA notes that VOC concentrations can be higher indoors than outdoors and can be influenced by product use and ventilation conditions [4]. Aerosol spray usage is therefore a relevant indoor activity to monitor because it can create short-term changes in gas-related and particle-related sensor readings.

Carbon dioxide is another important indoor measurement. Although CO_2 is not normally treated as a direct pollutant at typical indoor levels, increased indoor CO_2 can indicate reduced ventilation, higher occupancy, or limited air exchange with the outdoor environment [5]. Monitoring CO_2 together with particulate matter, temperature, humidity, and gas-resistance data can provide a wider understanding of the indoor environment.

Traditional air-quality monitoring systems often use fixed thresholds. These systems usually trigger an alarm when a single measurement exceeds a predefined limit. Although useful for basic monitoring, they do not always identify the cause of an air-quality change. For example, cooking and aerosol spray events may both increase particle concentration or affect gas-related readings, even though the sources are different. Previous research explains that air-quality data often involves several pollutant and environmental variables, and that machine-learning methods can identify complex relationships within structured sensor datasets [6].

Therefore, because indoor activities can produce overlapping sensor responses, a classification-based approach may be more useful than fixed thresholds.

This project investigates whether a low-cost embedded device can be made context-aware by classifying indoor conditions in real time. Machine learning is suitable because indoor environmental data includes several related variables that change over time. A practical method is TinyML, where a lightweight machine-learning model is deployed directly onto a microcontroller for local inference.

1.2 Current State of the Art

A key development in embedded environmental monitoring is the use of TinyML. TinyML allows machine-learning models to run on resource-constrained microcontrollers instead of relying on cloud processing. This can reduce latency, improve privacy, and allow the system to operate in real time with limited memory, processing power, and energy. Ray describes TinyML as a shift from high-power machine-learning systems towards low-end edge devices, while also highlighting challenges such as accuracy, reliability, and hardware limitations [7].

Recent work has shown that TinyML can be applied to air-quality monitoring. Wardana et al. demonstrated that TinyML models can be deployed on a low-cost air-quality monitoring device to perform prediction and data-imputation tasks on a microcontroller [8]. Platforms such as Edge Impulse also support the training, testing, optimisation, and deployment of embedded machine-learning models to microcontroller platforms.

However, many low-cost air-quality systems still rely mainly on fixed thresholds. These systems are simple and affordable, but they provide limited information about the type of event taking place. Cooking, aerosol spray use, cleaning product use, and poor ventilation may affect several measurements at the same time and may produce similar sensor responses. As a result, a fixed-threshold system may detect that an environmental change has occurred, but it may not reliably classify the source.

1.3 Problem Statement

The problem addressed in this project is the need for a low-cost embedded system that can measure indoor environmental conditions and classify the type of condition in real time. Existing threshold-based systems can detect when a measurement increases above a set value, but they may not provide enough context to distinguish between different indoor activities. This project therefore focuses on developing a prototype system that uses multiple

environmental sensors and TinyML classification to identify normal background, cooking, and aerosol spray conditions.

1.4 Aim and Objectives

The aim of this project is to design, implement, and evaluate a TinyML-based embedded indoor environmental monitoring system that can classify normal background, cooking, and aerosol spray conditions in real time using data from multiple sensors.

The objectives of this project are:

- To investigate indoor air-quality changes during normal background, cooking, and aerosol spray conditions.
- To develop an STM32-based embedded device for collecting environmental sensor data.
- To collect and label datasets for the selected indoor conditions.
- To prepare the collected data for machine-learning training.
- To train and evaluate a TinyML model using Edge Impulse.
- To deploy the trained model onto the STM32 platform for real-time classification.
- To assess the system performance, constraints, and classification results.
- To discuss the limitations of the system and identify possible improvements.

1.5 Scope and Limitations

This project focuses on the development of a prototype rather than a certified commercial air-quality monitor. The system is designed to classify three conditions only: normal background, cooking, and aerosol spray usage. These classes were selected because they represent common indoor activities and can produce measurable changes in environmental sensor readings.

The system does not directly measure every possible indoor pollutant. It should not be considered a professional regulatory monitoring device, a medical device, or a safety-critical alarm system. The purpose is to classify indoor conditions based on sensor patterns over time, rather than to determine whether legal exposure limits have been exceeded.

Several technical issues may also affect the system. These include sensor stability, reliable sensor integration, dataset size, labelling accuracy, and the delay between an activity ending and the sensor readings returning to normal. For example, particle or gas readings may remain elevated for several minutes after cooking or aerosol spray usage, making accurate

labelling more difficult. The deployed model must also fit within the memory and processing constraints of the STM32 microcontroller.

1.6 Possible Applications

A TinyML-based indoor monitoring system could be used as an air-quality awareness device in homes, kitchens, offices, and shared indoor spaces. It could provide ventilation feedback, support smart kitchen monitoring, or help users understand how everyday activities affect indoor air quality. With a larger dataset, similar systems could also be useful in busy indoor environments. It can also be used as hazard monitoring system to detect the signs of early hazards if trained with proper data.

1.7 Project Contribution

The main contribution of this project is the design and implementation of a low-cost STM32-based indoor environmental monitoring prototype that uses TinyML for real-time classification. The project also contributes a labelled dataset collected under normal background, cooking, and aerosol spray conditions. This dataset is used to evaluate whether low-cost sensors and embedded machine learning can distinguish between overlapping indoor events.

2 TECHNICAL WORK

2.1 Literature Review

Wardana et al. [8] demonstrate how TinyML can be used to improve low-cost air quality monitoring by deploying machine learning models directly on a resource-constrained microcontroller. Their work is important because it shows that embedded devices can support predictive and intelligent environmental monitoring while maintaining low power consumption, low latency, and reduced dependence on cloud processing. The system used sensor data such as CO_2, temperature, humidity, and electrical power parameters, collected over a three-month period at 10-minute intervals. The data was pre-processed, standardised, and scaled so that the models could learn useful patterns despite sensor variation, noise, or missing readings.

A key feature of Wardana et al.'s work is the use of two TinyML models on a single microcontroller. One model acted as a predictor for estimating air quality and power-related parameters, while the second model worked as an imputer to reconstruct missing sensor values. This is relevant to indoor hazard monitoring because real sensor systems may

experience incomplete data, noisy readings, or temporary communication issues. Their use of a 2D convolutional neural network for prediction and a denoising autoencoder for missing data reconstruction shows that advanced machine learning techniques can be adapted for low-power embedded systems. The study also applied post-training quantisation when converting TensorFlow models to TensorFlow Lite, reducing model size by up to 56.6% without major performance loss. This supports the feasibility of using TinyML on microcontrollers where RAM, flash memory, and processing power are limited.

Another relevant study by Arthur et al. [9] compares Edge Impulse and TensorFlow as TinyML development platforms for maize leaf disease identification. Although the application is agricultural image classification rather than indoor air quality monitoring, the paper is relevant because it evaluates the practical workflow required to collect data, train a model, test performance, and deploy inference on microcontroller-based hardware. The study used a dataset of 12,344 raw maize leaf images and 7,454 augmented images across five classes, showing the importance of a clearly labelled and structured dataset for classification tasks. This directly relates to this project because the indoor monitoring model also depends on correctly labelled classes such as normal, cooking, and aerosol spray.

Arthur et al. [9] found that TensorFlow offers greater flexibility and control over model architecture, training parameters, and optimisation. However, it requires more coding knowledge and has a steeper learning curve. Edge Impulse, in contrast, provides a more user-friendly workflow for data collection, preprocessing, training, testing, and deployment. This is useful for embedded TinyML projects because successful deployment is just as important as model accuracy. Their results showed that TensorFlow achieved slightly higher testing accuracy of 96.38%, compared with 94.84% for Edge Impulse. However, Edge Impulse required less memory, using 726.6 KB RAM and 344.7 KB flash compared with 890.5 KB RAM and 374.4 KB flash for TensorFlow. This highlights an important embedded-system trade-off: the most accurate model is not always the most suitable if it requires more memory, processing time, or deployment effort.

Both studies show that TinyML is suitable for real-time monitoring systems where local inference, low latency, and efficient resource use are required. Wardana et al. [8] provide evidence that low-cost sensors and TinyML can be combined for context-aware environmental monitoring, while Arthur et al. [9] highlight the practical trade-offs between development platforms such as TensorFlow and Edge Impulse. Together, these studies support the direction of this project, which aims to develop a microcontroller-based indoor

environmental hazard monitoring system using multiple sensors and TinyML classification. The literature suggests that sensor fusion, careful dataset preparation, model optimisation, and embedded deployment are key requirements for producing a reliable and practical system.

2.2 Overall System Architecture

The final system combined multi-sensor environmental sensing, STM32-based data acquisition, serial data logging, dataset preparation, model training, and embedded classification. The system was designed to detect environmental changes caused by indoor activities and classify them into three conditions: normal background conditions, cooking activity, and aerosol spray usage.

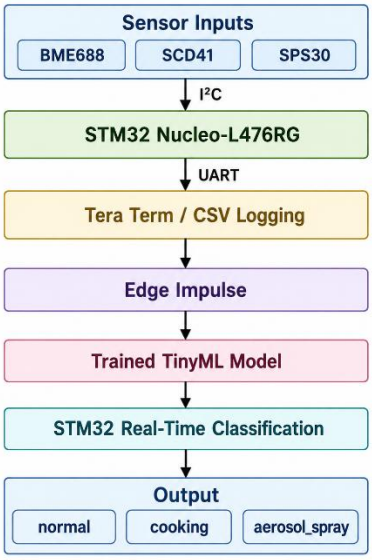

Figure 2-1: System architecture of the TinyML-based indoor environmental monitoring system

2.3 Sensor Selection

The sensors were selected based on measurement relevance, accuracy, communication interface, firmware support, and compatibility with the STM32 microcontroller. The aim was to collect environmental features suitable for classifying normal, cooking, and aerosol spray conditions.

2.3.1 Bosch BME688 Environmental and Gas Sensor

The Bosch BME688 was selected because it combines temperature, humidity, pressure, and gas-resistance sensing in a compact digital device [10]. This reduced hardware complexity while providing several useful environmental measurements.

Temperature and humidity were relevant because cooking can produce heat and moisture. Gas resistance was included because it responds to changes in the surrounding gas composition, which may occur during cooking or aerosol spray use. However, the BME688 does not directly measure individual VOC concentrations [10]. Therefore, gas resistance was treated as a general gas-response feature rather than a direct VOC measurement. Pressure was also recorded, although it was not expected to be a main indicator for the target activities.

2.3.2 Sensirion SCD41 Carbon Dioxide Sensor

The Sensirion SCD41 was selected to provide carbon dioxide measurement. CO_2 concentration is commonly used as an indicator of ventilation quality and indoor occupancy [11]. The sensor uses photoacoustic sensing based on non-dispersive infrared principles and provides an indoor CO_2 range suitable for embedded environmental monitoring [12].

In this project, CO_2 was mainly used to provide ventilation-related context during classification. Although the SCD41 also outputs temperature and humidity, these were not used as primary features because the datasheet gives slower response times of approximately 90 seconds for humidity and 120 seconds for temperature[12].

2.3.3 Sensirion SPS30 Particulate Matter Sensor

The Sensirion SPS30 was selected to measure airborne particulate matter, which is a major component of indoor air pollution. It provides PM1.0, PM2.5, PM4.0, and PM10 mass concentration outputs, as well as particle number concentration measurements [13].

These measurements were important because cooking was expected to increase particulate concentration, while aerosol spray events could produce different particle patterns depending on droplet size and ventilation conditions.

2.3.4 Multi-Sensor Fusion Justification

The selected sensors provide complementary measurements for indoor event classification. The BME688 captures environmental and gas-response changes, the SCD41 provides CO_2-based ventilation context, and the SPS30 detects particulate behaviour. Together, these features support classification more effectively than a single-sensor system.

The system was designed for event classification rather than laboratory-grade pollutant measurement. Therefore, the sensors were considered suitable because their operating ranges match expected domestic indoor conditions.

Sensor	Temperature Range	Humidity Range	Main Measurement Range
BME688	-40 to +85 °C	0–100% RH	Pressure: 300–1100 hPa
SCD41	-10 to +60 °C	0–95% RH	CO_2: 400–5000 ppm
SPS30	-10 to +60 °C	0–95% RH	PM mass: 0–1000 µg/m³

Table 2-1: Sensor Operating Ranges

Sensor	Main measurements used	Contribution to classification
BME688	Temperature, humidity, pressure, gas resistance	Detects environmental and gas-response changes
SCD41	CO_2 concentration	Provides ventilation and occupancy-related context
SPS30	PM mass, particle number, typical particle size	Detects airborne particulate behaviour

Table 2-2: Sensor role in the sensing platform

2.4 Hardware Platform and Sensor Integration

Several factors were considered when selecting the microcontroller, including sensor compatibility, processing capability, memory, low-power operation, TinyML support, development tools, library availability, cost, and accessibility.

2.4.1 Microcontroller Selection

The STM32 Nucleo-L476RG was selected because it provided a suitable balance between performance, memory, development support, and embedded deployment capability. Its Arm Cortex-M4 processor provided sufficient processing capacity for continuous environmental monitoring and TinyML inference. The board also offered suitable flash and RAM for the firmware and deployed machine-learning model.

The STM32 platform was also compatible with STM32CubeIDE, STM32CubeMX, HAL libraries, and Edge Impulse, which supported firmware development, debugging, and model deployment.

2.4.2 Electrical Integration and Pin Configuration

The sensors were connected to the STM32 using a shared I²C bus, while UART was used to transmit live readings to a computer for monitoring and dataset logging through Tera Term.

Component	Supply voltage	Communication	Address	Purpose
BME688	3.3 V	I²C	0x77	Temperature, humidity, pressure, gas resistance
SCD41	3.3 V	I²C	0x62	CO_2 concentration
SPS30	5 V	I²C	0x69	Particulate matter and particle number concentration
PC / Tera Term	USB/UART	UART	N/A	Serial monitoring and data logging

Table 2-3: Final Sensor Connections

The BME688 and SCD41 were powered from the 3.3 V rail, while the SPS30 was powered from the 5 V output due to its internal fan and optical measurement system. A short cable was used for the SPS30 connection to reduce noise and communication errors[13].

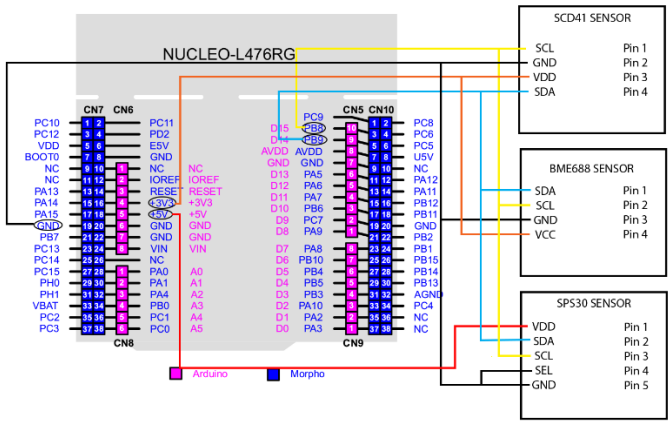

Figure 2-2: Wiring diagram of the STM32 Nucleo-L476RG sensor system

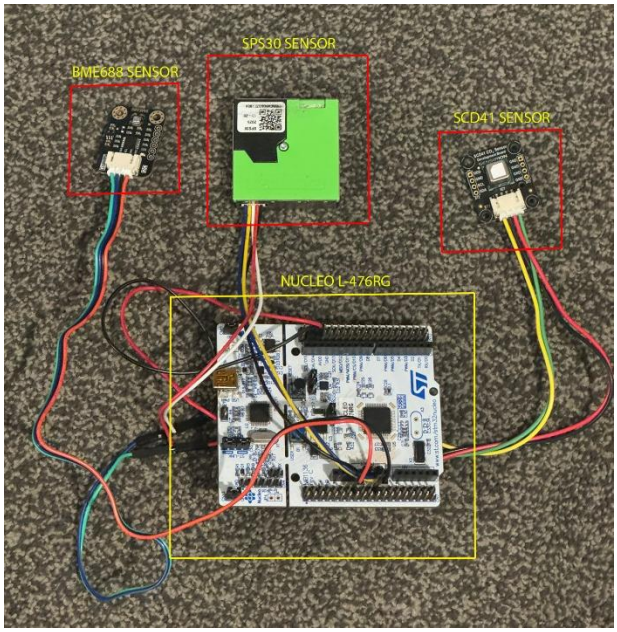

Figure 2-3: Physical prototype of the TinyML indoor environmental monitoring system

During initial testing, the default STM32CubeMX I²C pin configuration did not match the physical wiring, which prevented sensor detection. This was resolved by configuring PB8 as SCL and PB9 as SDA.

Parameter	Value used	Purpose
I²C SDA	PB9	Shared sensor data line
I²C SCL	PB8	Shared sensor clock line
I²C speed	100 kHz	Stable standard-mode communication
UART	USART2, 115200 baud	Serial output to Tera Term

Table 2-4: STM32 communication configuration

2.5 Firmware Design and Data Acquisition

The firmware was developed in STM32CubeIDE, with STM32CubeMX used to configure peripherals and generate the initial HAL-based project structure.

2.5.1 Firmware Architecture and Sensor Driver Integration

The firmware followed a sequential start-up structure. This ensured that each sensor was detected, configured, and started before data logging began. UART messages were printed at each stage to support debugging and confirm successful initialisation.

After HAL_Init(), the firmware configured the system clock, GPIO, I²C, UART, and RTC peripherals. I²C was used for the BME688, SCD41, and SPS30 sensors, while USART2 transmitted debugging messages and sensor readings to Tera Term at 115200 baud.

Official sensor drivers were integrated as separate files. The BME688 used the Bosch bme68x driver, the SCD41 used Sensirion scd4x_i2c and sensirion_i2c_hal functions, and the SPS30 used the Sensirion SPS30 I²C driver. This modular structure separated the main program from lower-level sensor communication.

2.5.2 Sensor Initialisation and Sampling

At start-up, the firmware first performed an I²C scan using HAL_I2C_IsDeviceReady(). This checked all 7-bit I²C addresses from 1 to 126 and printed detected device addresses through UART. This confirmed whether the sensors were correctly connected and powered.

The firmware then initialised the BME688, SCD41, and SPS30 sensors, stopped any previous SCD41 measurement mode, started SCD41 periodic measurement, woke the SPS30, and started SPS30 measurement in floating-point mode. A 15-second delay was included before logging began to reduce unstable start-up readings.

A 5000 ms sampling interval was implemented using HAL_GetTick(). This produced one structured data row every five seconds. The interval was suitable because indoor air-quality events such as cooking and aerosol spray usage change over seconds rather than milliseconds. It also matched the slower update rate of the SCD41.

During each acquisition cycle, the firmware read SPS30 particulate data, performed a BME688 measurement, checked whether SCD41 data were ready, and then read the CO_2 value. The readings were combined into one structured UART output row.

2.5.3 UART Logging and Serial Data Output

Tera Term was used to monitor, timestamp and save the UART output from the STM32. The firmware transmitted readings in a fixed comma-separated format so that each acquisition cycle produced one structured row.

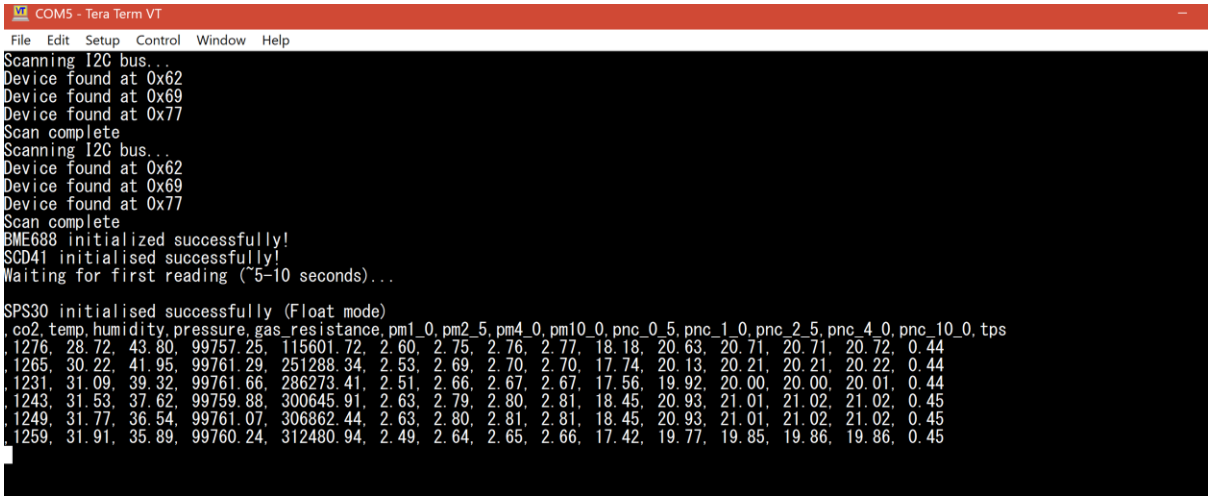

Figure 2-4: Tera Term output reading sensor values

T9

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
1	[2026-05-14 04:36:21.814] Scanning I2C bus...															
2	[2026-05-14 04:36:21.835] Device found at 0x62															
3	[2026-05-14 04:36:21.840] Device found at 0x69															
4	[2026-05-14 04:36:21.845] Device found at 0x77															
5	[2026-05-14 04:36:21.849] Scan complete															
6	[2026-05-14 04:36:23.741] BME688 initialized successfully!															
7	[2026-05-14 04:36:24.243] SCD41 initialized successfully!															
8	[2026-05-14 04:36:24.243] Waiting for first reading (~5-10 seconds)...															
9	[2026-05-14 04:36:24.243]															
10	[2026-05-14 04:36:24.285] SPS30 started successfully (Float mode)															
11	[2026-05-1	co2	temp	humidity	pressure	gas_resista	pm1_0	pm2_5	pm4_0	pm10_0	pnc_0_5	pnc_1_0	pnc_2_5	pnc_4_0	pnc_10_0	tps
12	[2026-05-1	465	25.03	37.97	99286.51	101748.8	2.2	2.77	3.14	3.33	14.4	17.05	17.51	17.58	17.59	0.6
13	[2026-05-1	482	24.96	37.67	99287.53	194899.1	2.22	2.79	3.15	3.34	14.51	17.17	17.64	17.7	17.71	0.6
14	[2026-05-1	488	25.78	35.99	99289.08	218197.3	2.06	2.61	2.97	3.16	13.38	15.89	16.34	16.41	16.42	0.6
15	[2026-05-1	468	26.06	34.74	99286.4	230734.6	2.03	2.56	2.91	3.09	13.23	15.68	16.12	16.18	16.19	0.6
16	[2026-05-1	469	26.25	33.97	99287.19	242366.9	1.89	2.38	2.7	2.86	12.35	14.63	15.03	15.09	15.1	0.6
17	[2026-05-1	465	26.16	33.37	99287.94	253152.1	2.25	2.84	3.22	3.42	14.64	17.37	17.85	17.92	17.93	0.6
18	[2026-05-1	465	26.16	33.01	99286.77	258912.8	2.2	2.77	3.13	3.32	14.38	17.03	17.49	17.56	17.57	0.59
19	[2026-05-1	463	26.04	32.86	99288.58	269119.6	2.06	2.57	2.9	3.07	13.5	15.96	16.37	16.43	16.44	0.59
20	[2026-05-1	464	26.06	32.69	99287.42	275416.9	2.09	2.63	2.98	3.16	13.63	16.16	16.6	16.66	16.67	0.6
21	[2026-05-1	465	26	32.62	99290.37	278563.7	1.96	2.49	2.84	3.02	12.71	15.11	15.55	15.61	15.62	0.6
22	[2026-05-1	465	26.02	32.73	99289.62	282951.1	1.87	2.38	2.72	2.89	12.16	14.45	14.87	14.93	14.94	0.6
23	[2026-05-1	466	26.01	32.7	99290.54	288694.7	1.8	2.28	2.6	2.77	11.69	13.88	14.28	14.34	14.35	0.6
24	[2026-05-1	466	26.04	32.67	99288.72	293409.8	1.86	2.39	2.73	2.91	12.09	14.39	14.82	14.88	14.89	0.61

Figure 2-5: Logged dataset CSV format

The firmware output was kept as raw serial sensor data to reduce microcontroller processing load. Any later cleaning, formatting, and labelling were completed outside the firmware stage.

2.5.4 Firmware Validation

The firmware was validated in stages. First, the STM32 project was compiled and flashed successfully. UART output was then tested using printf() through USART2. The I²C scan was used to confirm that the sensors were visible on the shared bus.

After sensor detection, each sensor was checked using live readings. Final validation involved observing sensor responses during normal background conditions, cooking, and aerosol spray experiments. The readings changed as expected, confirming that the system was suitable for repeatable data acquisition.

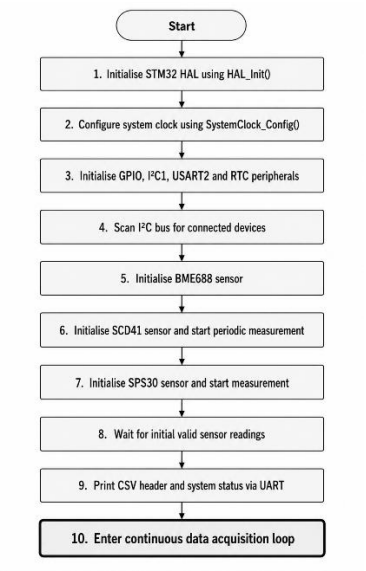

Figure 2-6: Firmware initialisation

2.6 Dataset Collection Procedure

The aim of dataset collection was to obtain realistic sensor data for training and testing a TinyML model to classify three indoor conditions: normal background, cooking, and aerosol spray. Data were collected under different activity and ventilation conditions to improve dataset variety.

2.6.1 Experimental Environment and Sensor Placement

Dataset collection was carried out in a domestic kitchen measuring 3.96 m × 3.58 m. The sensing platform was positioned 0.86 m above the floor, while the induction hob height was 0.90 m. The diagonal distance between the sensor platform and the hob was 0.89 m. Although this was not a final practical mounting position, it provided a controlled and repeatable location for experimental data collection.

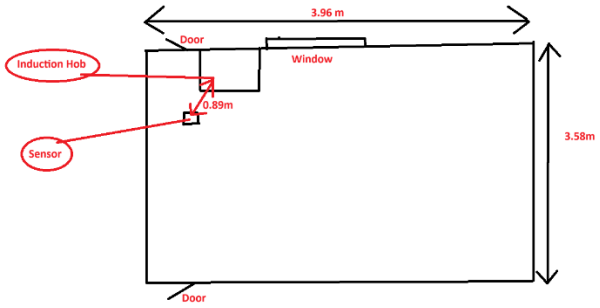

Figure 2-7: Kitchen Mockup

An induction hob was used for all cooking experiments. Unlike gas appliances, induction cooking does not directly produce combustion emissions. Therefore, the measured changes

during cooking were mainly associated with cooking aerosols, steam, oil vapour, heat, and particulate generation. During each session, the kitchen extractor was kept off and internal doors remained closed to reduce uncontrolled airflow. The same kitchen, sensor position, hob, sampling interval, sensor setup, and logging method were used across all sessions.

2.6.2 Dataset Sessions and Experimental Conditions

Six datasets were collected under different activity and ventilation conditions.

Dataset	Activity	Window state	Time period	Label
D1	Normal background	Open	Night	normal
D2	Normal background	Closed	Night	normal
D3	Cooking	Open	Day/evening	cooking
D4	Cooking	Closed	Day/evening	cooking
D5	Aerosol spray	Open	Day/evening	aerosol
D6	Aerosol spray	Closed	Day/evening	aerosol

Table 2-5: Dataset collection conditions

For cooking and aerosol sessions, recording began under normal background conditions. The target activity was introduced approximately 10 to 15 minutes later. The normal datasets were used to establish baseline kitchen conditions when no major pollution-generating activity was present.

The cooking datasets involved frying steak with butter for approximately 10 to 15 minutes. This activity was selected because it produces realistic cooking-related changes, including fine particles, oil vapour, heat, and moisture.

The aerosol datasets were collected using deodorant spray and air freshener. Both products were grouped into one aerosol class because the aim was to detect short aerosol-related events rather than classify individual products. Data collection continued until readings returned close to normal background levels.

2.6.3 Sampling Interval and Sensor Stabilisation

Data were sampled every five seconds. This interval captured gradual indoor air-quality changes while limiting data redundancy and file size. It also matched the SCD41 update rate, which provides new readings approximately every five seconds.

The first 10 minutes of readings were discarded after system start-up to reduce the effect of unstable sensor behaviour and improve dataset consistency.

2.6.4 Data Logging, Cleaning, and Labelling

Raw data were collected from the STM32 through UART using TeraTerm. Timestamps and CSV formatting were added during serial logging by TeraTerm. The recorded data were then cleaned and labelled for machine learning preparation. The label column was used by Edge Impulse as the target output for supervised machine learning.

Each CSV file was labelled according to the dominant experimental condition. For cooking and aerosol datasets, the label included the active event period and the recovery period where sensor readings remained clearly elevated. This matched the practical aim of detecting recent or ongoing environmental events rather than only identifying the exact moment when the source was active.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
1		co2	temp	humidity	pressure	gas_resista	pm1_0	pm2_5	pm4_0	pm10_0	pnc_0_5	pnc_1_0	pnc_2_5	pnc_4_0	pnc_10_0	tps	label
2	[2026-05-0	516	27.2	31.83	100570.6	272123.3	6.22	6.57	6.57	6.57	43.62	49.44	49.61	49.62	49.63	0.42	normal
3	[2026-05-0	516	27.17	31.86	100572.4	271906.5	6.17	6.52	6.52	6.52	43.27	49.05	49.22	49.23	49.24	0.42	normal
4	[2026-05-0	517	27.18	31.92	100571.3	273431.3	6.11	6.46	6.46	6.46	42.89	48.62	48.79	48.8	48.81	0.42	normal
5	[2026-05-0	519	27.16	31.96	100573.4	271042.9	5.91	6.25	6.25	6.25	41.44	46.97	47.14	47.15	47.16	0.43	normal
6	[2026-05-0	517	27.19	32	100571.2	269971	6.04	6.38	6.38	6.38	42.35	48	48.17	48.18	48.19	0.43	normal
7	[2026-05-0	516	27.22	32.01	100571.3	269119.6	5.87	6.21	6.21	6.21	41.18	46.68	46.85	46.86	46.86	0.42	normal
8	[2026-05-0	515	27.26	31.99	100572.5	268484.5	5.83	6.16	6.16	6.16	40.89	46.35	46.51	46.52	46.52	0.42	normal
9	[2026-05-0	516	27.28	31.96	100572.2	269331.9	5.91	6.25	6.25	6.25	41.48	47.02	47.19	47.2	47.2	0.42	normal
10	[2026-05-0	516	27.31	31.93	100572.1	269757.6	6.03	6.37	6.37	6.37	42.27	47.91	48.09	48.1	48.1	0.42	normal
11	[2026-05-0	516	27.33	31.91	100569.3	269757.6	5.98	6.32	6.32	6.32	41.96	47.56	47.73	47.74	47.75	0.41	normal
12	[2026-05-0	517	27.34	31.87	100571	268273.5	5.96	6.3	6.3	6.3	41.83	47.41	47.58	47.59	47.59	0.41	normal
13	[2026-05-0	517	27.36	31.83	100570.3	268273.5	5.91	6.25	6.25	6.25	41.46	46.99	47.16	47.17	47.18	0.41	normal
14	[2026-05-0	518	27.34	31.8	100569.9	269544.6	5.56	5.88	5.88	5.88	39.01	44.22	44.38	44.39	44.39	0.42	normal
15	[2026-05-0	517	27.32	31.76	100569.7	270398.7	5.4	5.71	5.71	5.71	37.88	42.94	43.09	43.1	43.1	0.42	normal
16	[2026-05-0	518	27.33	31.75	100570.1	269757.6	5.33	5.63	5.63	5.63	37.39	42.38	42.53	42.54	42.54	0.43	normal
17	[2026-05-0	517	27.32	31.78	100570.3	269757.6	5.19	5.49	5.49	5.49	36.45	41.31	41.46	41.47	41.47	0.42	normal
18	[2026-05-0	516	27.32	31.78	100570.2	268695.9	5.23	5.53	5.53	5.53	36.7	41.6	41.75	41.75	41.76	0.42	normal
19	[2026-05-0	518	27.33	31.8	100570.7	269971	5.57	5.89	5.89	5.89	39.07	44.28	44.44	44.45	44.45	0.43	normal
20	[2026-05-0	516	27.3	31.77	100570.3	272123.3	5.94	6.28	6.28	6.28	41.68	47.25	47.42	47.43	47.43	0.43	normal

Figure 2-8: Labelled and cleaned CSV dataset prepared for Edge Impulse

Class	Number of readings	Approx. duration
normal	5267	439 minutes
cooking	879	73 minutes
aerosol	352	29 minutes

Table 2-6: Final labelled dataset summary

2.6.5 Recovery Period and Label Assignment

After cooking stopped, the sensor readings did not immediately return to normal. A recovery period of approximately 15 to 20 minutes was observed, although this was shorter in aerosol and when windows were open. During this time, particulate matter, humidity, gas response, and other environmental indicators gradually returned towards background levels.

Early recovery samples were labelled as cooking while the sensor response remained clearly elevated. Once readings approached the normal baseline, samples were labelled as normal. Ambiguous transition samples were excluded where necessary to reduce label noise.

The same approach was used for aerosol recordings. Although the spray event was short, sensor readings remained affected after spraying. Samples were labelled as aerosol while the response remained clearly different from normal background conditions.

2.7 TinyML Model Development and Training

The TinyML model was developed in Edge Impulse using the cleaned and labelled sensor dataset. The aim was to train a lightweight classifier for three indoor conditions: normal, cooking, and aerosol.

The impulse used a time-series input block with 15 sensor axes. A 60,000 ms window size and 60,000 ms window increase were selected. Since the sampling interval was five seconds, each window contained 12 readings. This produced 180 input features per inference window.

A Raw Data processing block was used because the input values were already structured environmental measurements. StandardScaler normalisation was applied to reduce the effect of different feature scales, such as pressure, gas resistance, CO_2, and particulate measurements.

The classifier used a neural network with an input layer of 180 features, followed by dense layers of 20 and 10 neurons. The output layer contained three classes: aerosol, cooking, and normal. The model was trained for 100 cycles, with a learning rate of 0.0005, batch size of 32, and a validation split of 20%.

The model was profiled as an int8 quantised neural network. This was suitable for TinyML deployment because int8 quantisation reduces memory use and processing demand compared with a float32 model. This allowed the trained classifier to be deployed to the STM32 microcontroller for local inference.

2.8 Embedded Deployment and Real-Time Classification

After training, the model was exported from Edge Impulse as an embedded C/C++ library and added to the STM32CubeIDE project. The existing sensor acquisition firmware was then extended so that live readings could be passed into the classifier.

The live input format had to match the training format. Each sensor cycle produced 15 values, which were stored in a fixed buffer.

Once the buffer was full, the values were passed to the Edge Impulse classifier. The model returned probability scores for the three classes, and the firmware selected the class with the highest score as the predicted condition. The predicted class and confidence values were printed through UART and viewed on a serial terminal.

Stage	Function
Sensor acquisition	Read live BME688, SCD41, and SPS30 data
Buffering	Store readings
Inference	Run the Edge Impulse classifier on STM32
Output	Print predicted class through UART

Table 2-7: Embedded real-time inference workflow

The classifier successfully ran on the STM32 prototype. This confirms that the project was not limited to offline model training; the trained TinyML model was deployed for local embedded inference.

2.9 Embedded Resource Usage

The model was also checked using the Edge Impulse deployment/profile information.

Deployment metric	Quantised int8 result
Model format	Quantised int8 neural network
Estimated latency	1 ms
Total RAM usage	1.6K
Classifier RAM usage	1.0K
Raw data RAM usage	0.6K
Classifier flash / ROM usage	17.8K
Reported accuracy	98.02%
Edge Impulse target used for estimate	ST STM32N6 Cortex-M55 800 MHz

Table 2-8: Edge Impulse int8 deployment profiling results

The profiling results show that the model is lightweight for embedded TinyML deployment. The quantised int8 version required only 1.6K RAM and 17.8K flash/ROM for the classifier, with an estimated inference latency of 1 ms on the selected Edge Impulse STM32N6 target.

Although this target is not identical to the STM32 Nucleo-L476RG used in the project, the result still provides useful evidence that the model has low memory and processing requirements.

2.10 System Performance Evaluation

2.10.1 Model Training and Validation Performance

The model achieved strong performance in Edge Impulse. The training accuracy was approximately **99.06%**, while the int8 validation accuracy was **100%** with a validation loss of **0.0202**.

Metric	Result
Total data collected	9 h 1 min 30 s
Train/test split	80% / 20%
Training samples	399
Test samples	101
Training windows	398
Number of classes	3
Classes	aerosol, cooking, normal
Window size	60,000 ms
Input layer features	180
Training accuracy	99.06%
Validation accuracy	100%
Int8 validation loss	0.0202

Table 2-9: TinyML model training and validation results

These results show that the model separated the three classes well within the collected dataset. However, the result should not be overgeneralised because the data were collected using one prototype setup and a limited set of indoor conditions.

2.10.2 Confusion Matrix Analysis

Actual class	Predicted aerosol	Predicted cooking	Predicted normal
Aerosol	3	0	0
Cooking	0	10	0
Normal	0	0	67

Table 2-10: Validation confusion matrix

Figure 2-9: Edge Impulse confusion matrix for the trained int8 classifier

The confusion matrix shows that all validation samples were classified correctly. The model correctly identified 67 normal, 10 cooking, and 3 aerosol windows. In the validation set, all samples were classified correctly. However, the separate held-out test set showed two aerosol windows being classified as cooking. This suggests that the model performs strongly overall, but aerosol detection is less robust than normal and cooking classification because the aerosol class has fewer examples.

This is a positive result because cooking and aerosol spray can both affect particle-related readings. The result suggests that the model used the combined multi-sensor pattern rather than relying on one measurement only. However, the aerosol class had only three validation samples, so more aerosol data would be needed to prove stronger reliability.

2.10.3 Real-Time Prototype Testing

The deployed model was tested using live sensor readings from the STM32 prototype.

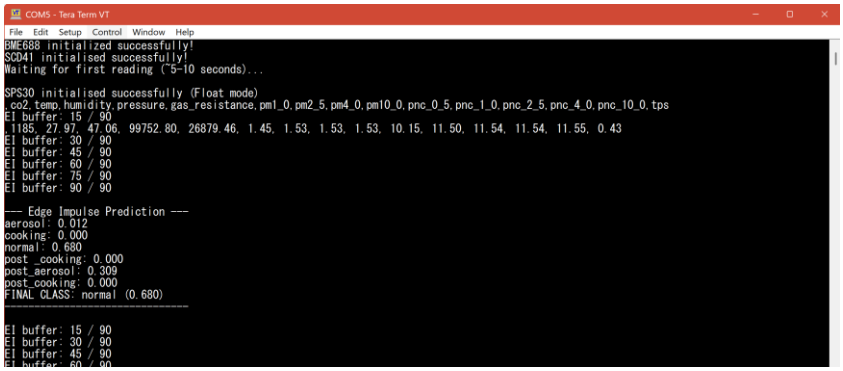

Figure 2-10: Tera Term live classification output from the final STM32 prototype

Test condition	Expected class	Prototype output	Result
Normal room	normal	normal	Correct
Cooking / frying	cooking	cooking	Correct
Aerosol spray	aerosol	aerosol	Correct

Table 2-11: Real-time prototype testing results

The prototype printed predictions through UART, allowing the output to be monitored in real time. Normal conditions were classified as **normal**, cooking as **cooking**, and aerosol spray as **aerosol**.

After spraying stopped, the system sometimes continued to output aerosol. This is reasonable because the sensors detect the air condition, not the user action itself. Aerosol particles and gas-related changes can remain in the air after spraying, especially with limited ventilation. Therefore, the continued aerosol output during recovery indicates that the air was still affected.

2.10.4 Dataset and Evaluation Limitations

The main limitation is dataset variety. The data were collected using one hardware setup and mainly one indoor environment. Room size, sensor position, ventilation, cooking method, and aerosol product type could all affect the readings.

The class balance was also limited. The normal class had much more data than the aerosol class, and the aerosol validation result was based on only three windows. This reduces confidence in the robustness of aerosol detection.

Labelling was another challenge because cooking and aerosol effects do not stop immediately when the activity stops. Recovery samples can still contain elevated readings, so exact label boundaries are difficult to define.

2.11 PCB Design Using KiCad

KiCad was used to design a custom printed circuit board (PCB) for the embedded monitoring system. The purpose of the PCB design was to improve integration, reliability, and organisation of the hardware compared with the initial wired prototype, which was mainly used for testing and debugging.

The PCB was designed to interface the STM32 microcontroller with the three environmental sensors used in the project. The design process included schematic capture, footprint assignment, component placement, and PCB routing. Net labels were used to improve schematic readability and reduce wiring complexity within the design. Connector headers were also included so that the sensors could be removed, replaced, or repositioned during testing without permanently soldering them to the board.

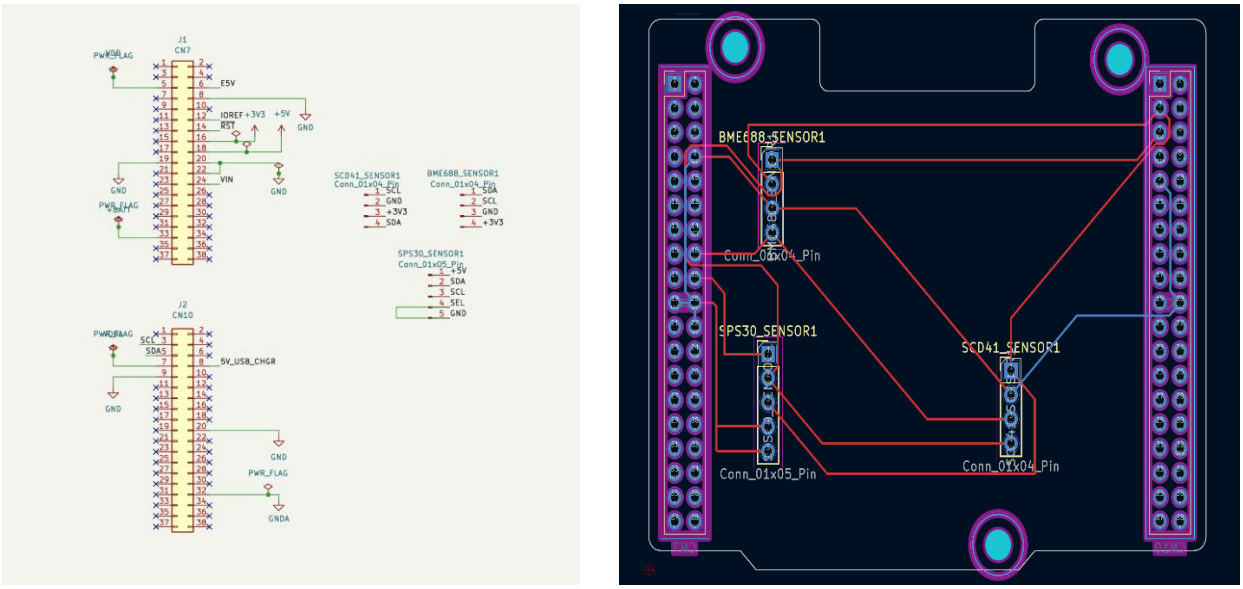

Figure 2-11: KiCad schematic design and PCB layout

Component placement was considered carefully to reduce routing complexity and minimise unnecessary trace lengths. Shorter traces helped produce a cleaner layout and reduced the risk of unwanted noise pickup on signal lines. Power distribution was also considered because the system used both 3.3 V and 5 V supplies. The BME688 and SCD41 were supplied from 3.3 V, while the SPS30 required a 5 V supply.

Although the PCB was not manufactured within the project timescale, the design was checked using KiCad's Electrical Rule Check and Design Rule Check tools. Both checks passed with 0 errors, indicating that the schematic connections and PCB layout met the defined design constraints. However, because the board was not physically fabricated, practical validation such as assembly testing, signal verification, and long-term reliability testing could not be completed.

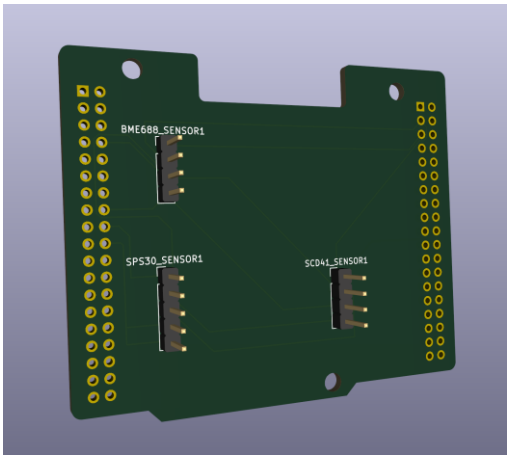

Figure 2-12: KiCad 3D view of the proposed PCB design

PCB design provided a practical foundation for future hardware development if the system were to be manufactured and tested as a complete standalone device.

2.12 Professional, Ethical, Risk, Security and Inclusion Considerations

Professional, ethical, risk, security, and inclusion factors were considered because the system monitors indoor environmental conditions that may influence user decisions about ventilation and air quality. The project has a positive societal purpose because it supports awareness of indoor changes caused by common activities such as cooking and aerosol spray usage. By running the TinyML model locally on the STM32, the system also reduces the need for cloud processing, which can lower network dependence and improve data privacy.

Ethical and safety considerations affected the project scope. More dangerous hazards such as fire, smoke, or gas leaks were not deliberately tested because they would create unnecessary safety risks. Instead, the project focused on safer and realistic indoor conditions: normal background, cooking, and aerosol spray. The system is therefore presented as a prototype for environmental classification, not as a certified safety alarm, medical device, or regulatory air-quality monitor.

Several risks were managed during development. Sensor communication risk was reduced by testing each sensor separately, using an I²C scan, and validating live readings before data collection. Dataset risk was managed by collecting data under different conditions, removing unstable start-up readings, and excluding ambiguous transition samples where necessary. Labelling was also a challenge because cooking and aerosol effects remained in the air after the activity stopped. This was managed by labelling recovery periods according to the observed sensor response rather than only the action timing.

Deployment risk was considered because the model had to fit within STM32 memory and processing constraints. This was reduced by using an int8 quantised model and checking Edge Impulse profiling results. However, the full firmware memory usage should still be checked using the STM32CubeIDE build report because the complete system includes sensor drivers, HAL libraries, UART output, and application code.

Security and privacy risks were limited because the prototype used local inference and collected only environmental sensor readings. However, UART logs should still be handled carefully because they may reveal activity patterns. If the system were developed further with wireless connectivity, secure authentication, encryption, and clear data-handling rules would be required.

Inclusion was considered using low-cost, accessible hardware and simple output classes such as normal, cooking, and aerosol. Future versions should avoid relying only on Serial output and should provide clear text, symbols, or sound feedback. Further testing in different homes, room sizes, ventilation conditions, and cooking styles would also improve fairness and reliability across different users.

3 CONCLUSION

This project achieved its main aim of developing a TinyML-based embedded system for real-time classification of indoor environmental conditions. The final prototype combined environmental sensing, embedded firmware, dataset collection, machine-learning training, and microcontroller deployment into one working system. It demonstrated that a low-cost embedded platform could move beyond simple air-quality monitoring and provide more meaningful classification of indoor events.

The main finding is that indoor activities such as cooking and aerosol spray usage produce patterns across several environmental measurements rather than changes in only one

parameter. This made a multi-sensor and machine-learning approach more suitable than a fixed-threshold method. The system was able to use combined sensor behaviour to distinguish between normal background conditions, cooking, and aerosol spray. This supports the idea that TinyML can improve context awareness in small, embedded monitoring devices.

The collected dataset was used to train and validate a three-class model in Edge Impulse. The model achieved strong results within the collected dataset, with high training performance and correct classification of the validation samples. The confusion matrix showed that the model could separate the three target classes without misclassification during validation. This indicates that the selected features and time-window approach were appropriate for the classification task.

The model was also deployed successfully onto the STM32 microcontroller, which confirmed that the project was not only a software-based machine-learning exercise. Live sensor readings were collected, buffered, classified locally, and output through UART. This showed that real-time inference was possible on the embedded prototype. The low model memory usage also supported the suitability of the approach for resource-constrained hardware.

However, the results must be interpreted carefully. The dataset was collected in one domestic environment using one hardware arrangement and a limited number of test conditions. As a result, the trained model may not generalise reliably to all rooms, cooking styles, ventilation conditions, or aerosol products. The aerosol class was also smaller than the other classes, so further data collection would be needed to improve balance and reliability. Another limitation was the difficulty of labelling recovery periods, because the air remained affected after the activity itself had stopped.

Overall, the project shows that embedded TinyML can be used to classify indoor environmental conditions locally and effectively within a prototype system. The work provides a useful foundation for future development of context-aware indoor monitoring devices. Future improvements should include a larger and more balanced dataset, testing in different environments, comparison with threshold-based methods, full firmware memory analysis, and physical manufacture and validation of the PCB design.

4 REFLECTION

This project has been a significant learning experience because it developed from an uncertain initial idea into a more focused and technically challenging embedded TinyML system. At the beginning, my original project idea was an IoT greenhouse monitoring system. However, after discussions with my supervisor and reflection on my own interests and career direction, I decided to change the project to TinyML for Indoor Environmental Hazard Monitoring. This was an important turning point because it aligned the project more closely with embedded systems, machine learning, sensor integration, and real-world environmental monitoring.

One of the main challenges at the start was the lack of clarity in the project scope. I initially had a broad idea of detecting indoor hazards, but I did not fully understand what the system should classify, which sensors were required, or how TinyML could be applied practically. Through research and supervisor feedback, I gradually refined the project from a general hazard detection concept into a safer and more realistic context-aware classification system. This helped me understand the importance of defining measurable objectives rather than starting with a vague system outcome.

A major lesson I learned was the importance of project planning and risk management. Creating the Gantt chart helped me break the project into stages such as research, hardware selection, sensor integration, data collection, TinyML development, testing, and final documentation. Although the project direction changed early on and I faced delays due to illness, the planning process helped me recover and restructure the work. It also showed me that engineering projects rarely follow the original plan exactly, so flexibility and continuous adjustment are essential.

Technically, this project pushed me outside my comfort zone. I had limited experience with machine learning, STM32 development, and embedded sensor integration before starting. To address this, I spent time learning the basics of TinyML, STM32CubeIDE, STM32CubeMX, I2C communication, UART debugging, and sensor drivers. As the project progressed, I became more confident in configuring peripherals, testing communication with sensors, and debugging embedded software issues. For example, resolving problems such as I2C pin mapping and floating-point printing in UART output improved my understanding of low-level embedded constraints.

The sensor integration stage was one of the most valuable parts of the project. Working with sensors such as the BME688, SCD41, SPS30, and MLX90614 helped me understand how different environmental parameters can support classification. I learned that indoor air quality cannot be reliably judged from one variable alone. Instead, changes in temperature, humidity, CO_2, particulate matter, and gas resistance need to be considered together. This strengthened my understanding of sensor fusion and showed why TinyML can be more suitable than simple threshold-based detection in noisy indoor environments.

The project also improved my ability to critically evaluate design decisions. Initially, I considered direct hazard scenarios such as fire or gas leak detection, but supervisor feedback made me recognise the safety and ethical risks of simulating dangerous events. As a result, I refined the scope to focus on safe, observable indoor conditions such as normal air, cooking activity, and poor air quality indicators. This made the project safer, more realistic, and more academically appropriate. It also helped me understand that engineering decisions must consider not only technical feasibility but also user safety, ethics, and responsible deployment.

Another important learning outcome was the role of data quality in machine learning systems. I realised that the TinyML model would only be as reliable as the data used to train it. This made structured data collection a critical part of the project. I had to think carefully about sampling intervals, labels, environmental conditions, and consistency across readings. This changed my view of machine learning from simply “training a model” to a full engineering process involving sensing, preprocessing, feature selection, deployment constraints, and evaluation.

The project also developed my practical engineering skills beyond software. Learning KiCad and beginning a PCB design helped me understand how a prototype can move closer to a professional embedded system. Breadboard wiring was useful for testing, but it also introduced issues such as wiring complexity and reduced reliability. Designing a PCB made me think more carefully about component placement, routing, power distribution, and manufacturability. This was useful because it connected the project to real industry practices.

From a professional development perspective, the project helped me improve my independence and problem-solving skills. Many problems did not have immediate solutions, so I had to research datasheets, examine drivers, test hardware step by step, and make design decisions based on evidence. I also learned the importance of documenting progress through

weekly journal entries, as this made it easier to reflect on decisions, justify changes, and track development over time.

Overall, this project has improved my confidence in embedded systems, sensor-based design, and TinyML deployment. It taught me that successful engineering work requires more than building a working prototype; it also requires clear scope, safe design decisions, critical analysis, reliable data, and evidence-based evaluation. Although the project began with uncertainty and several changes, these challenges helped strengthen the final direction. The experience has given me practical skills that are relevant to embedded systems, manufacturing engineering, product development, and intelligent monitoring systems.

REFERENCES

[1] United States Environmental Protection Agency, “Introduction to Indoor Air Quality,” EPA, Mar. 19, 2026. [Online]. Available: https://www.epa.gov/indoor-air-quality-iaq/introduction-indoor-air-quality. Accessed: May 15, 2026.

[2] World Health Organization, WHO Global Air Quality Guidelines: Particulate Matter (PM2.5 and PM10), Ozone, Nitrogen Dioxide, Sulfur Dioxide and Carbon Monoxide. Geneva, Switzerland: World Health Organization, 2021. [Online]. Available: https://www.who.int/publications/i/item/9789240034228. Accessed: May 15, 2026.

[3] Health Canada, “Factsheet: Cooking and Indoor Air Quality,” Government of Canada, Jan. 12, 2021. [Online]. Available: https://www.canada.ca/en/health-canada/services/publications/healthy-living/factsheet-cooking-and-indoor-air-quality.html. Accessed: May 15, 2026.

[4] United States Environmental Protection Agency, “Volatile Organic Compounds’ Impact on Indoor Air Quality,” EPA, Jul. 24, 2025. [Online]. Available: https://www.epa.gov/indoor-air-quality-iaq/volatile-organic-compounds-impact-indoor-air-quality. Accessed: May 15, 2026.

[5] United States Environmental Protection Agency, “Can I Measure Carbon Dioxide (CO_2) Indoors to Get Information on Ventilation?” EPA, Aug. 13, 2025. [Online]. Available: https://www.epa.gov/indoor-air-quality-iaq/can-i-measure-carbon-dioxide-co2-indoors-get-information-ventilation. Accessed: May 15, 2026.

[6] L. Miranda, C. Duc, N. Redon, J. Pinheiro, B. Dorizzi, J. Montalvao, M. Verriele, and J. Boudy, “Automatic detection of indoor air pollution-related activities using metal-oxide gas sensors and the temporal intrinsic dimensionality estimation of data,” Indoor Environments, vol. 1, no. 3, Art. no. 100026, 2024, doi: 10.1016/j.indenv.2024.100026.

[7] P. P. Ray, “A review on TinyML: State-of-the-art and prospects,” Journal of King Saud University – Computer and Information Sciences, vol. 34, no. 4, pp. 1595–1623, 2022, doi: 10.1016/j.jksuci.2021.11.019.

[8] N. K. Wardana, J. W. Gardner, and S. A. Fahmy, “TinyML models for a low-cost air quality monitoring device,” *IEEE Sensors Letters*, vol. 7, no. 11, Art. no. 6007804, 2023.

[9] E. A. E. Arthur, F. A. Wulnye, D. A. N. Gookyi, K. O. B. O. Agyekum, P. Danquah, and R. Gyaang, “Edge Impulse vs TensorFlow: A comparative analysis of TinyML

platforms for maize leaf disease identification," in *2024 Conference on Information Communications Technology and Society (ICTAS)*, 2024, pp. 1–6, doi: 10.1109/ICTAS59620.2024.10507119.

[10] Bosch Sensortec, BME688 Datasheet: Digital Low Power Gas, Pressure, Temperature & Humidity Sensor with AI, Doc. BST-BME688-DS000-03, Rev. 1.3, Feb. 2024. [Online]. Available: https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bme688-ds000.pdf. Accessed: May 15, 2026.

[11] Health and Safety Executive, "Using CO_2 monitors – Ventilation in the Workplace." [Online]. Available: https://www.hse.gov.uk/ventilation/using-co2-monitors.htm. Accessed: May 15, 2026.

[12] Sensirion AG, Datasheet SCD4x CO_2 Sensor, Version 1.7, Apr. 2025. [Online].Available: https://sensirion.com/media/documents/48C4B7FB/67FE0194/CD_DS_SCD4x_Datasheet_D1.pdf. Accessed: May 15, 2026.

[13] Sensirion AG, Datasheet SPS30 Particulate Matter Sensor for Air Quality Monitoring and Control, Version 2.0, Jun. 2023. [Online]. Available: https://sensirion.com/media/documents/8600FF88/64A3B8D6/Sensirion_PM_Sensors_Datasheet_SPS30.pdf. Accessed: May 15, 2026.

PROJECT PORTFOLIO

This portfolio presents three review documents that were prepared and shared with the supervisor during the project. The content included here is a verbatim reproduction of the original submissions, with no modifications. All dates and information are accurate and correspond to the materials provided to the supervisor, as referenced on the second page of this final submission and repeated below for clarity. This section serves as a record of ongoing progress and documented feedback throughout the project.

Document Name	Date and Means of Approval
Review Document 1	19/11/2025, Approved verbally
Review Document 2	03/12/2025, Approved verbally
Review Document 3	03/02/2026, Approved verbally
Review Document 4	17/02/2026, Approved verbally
Review Document 5	05/03/2026, Approved verbally
Review Document 6	17/03/2026, Approved verbally
Review Document 7	24/03/2026, Approved verbally
Review Document 8	30/04/2026, Approved verbally
Review Document 9	08/05/2026, Approved verbally
Review Document 10	